

5. UML에 대한 다음 설명에서 괄호(①~③)에 들어갈 알맞은 용어를 쓰시오. 22-3

UML은 시스템 분석, 설계, 구현 등 시스템 개발 과정에서 시스템 개발자와 고객 또는 개발자 상호 간의 의사소통이 원활하게 이루어지도록 표준화한 대표적인 객체지향 모델링 언어로, 사물, (①), 다이어그램으로 이루어져 있다.

- (①)는 사물과 사물 사이의 연관성을 표현하는 것으로, 연관, 집합, 포함, 일반화 등 다양한 형태의 (①)가 존재한다.
- (②)는 UML에 표현되는 사물의 하나로, 객체가 갖는 속성과 동작을 표현한다. 일반적으로 직사각형으로 표현하며, 직사각형 안에 이름, 속성, 동작을 표기한다.
- (③)는 (②)와 같은 UML에 표현되는 사물의 하나로, (②)나 컴포넌트의 동작을 모아놓은 것이며, 외부적으로 가시화되는 행동을 표현한다. 단독으로 사용되는 경우는 없으며, (③) 구현을 위한 (②) 또는 컴포넌트와 함께 사용된다.

답: ① **관계(Relationship)** ② **클래스(Class)** ③ **인터페이스(Interface)**

9. 데이터베이스의 스키마(Schema)에 대해 간략히 서술하시오. 20-3

답: **스키마는 데이터베이스의 구조와 제약 조건에 관한 전반적인 명세를 기술한 것이다.**

12. 데이터 모델의 구성 요소에 대한 다음 설명에서 괄호(①, ②)에 들어갈 알맞은 용어를 쓰시오. 21-1

1. (①)은 데이터베이스에 저장된 실제 데이터를 처리하는 작업에 대한 명세로서 데이터베이스를 조작하는 기본 도구에 해당한다.

2. (②)는 논리적으로 표현된 객체 타입들 간의 관계로서 데이터의 구성 및 정적 성질을 표현한다.

3. 제약 조건은 데이터베이스에 저장될 수 있는 실제 데이터의 논리적인 제약 조건을 의미한다.

답: ① **연산(Operation)** ② **구조(Structure)**

24. 데이터베이스에서 비정규화(Denormalization)의 개념을 서술하시오. 20-1

답: **비정규화는 정규화된 데이터 모델을 통합, 중복, 분리하는 과정으로, 의도적으로 정규화 원칙을 위배하는 행위이다.**

25. 데이터베이스의 상태 변화를 일으키는 트랜잭션(Transaction)의 특성 중 원자성(Atomicity)에 대해 간략히 서술하시오. 21-2

답: **원자성은 트랜잭션의 연산은 데이터베이스에 모두 반영되도록 완료되든지 아니면 전혀 반영되지 않도록 복구되어야 한다는 특성을 의미한다.**

28. 시스템 관리와 관련하여 다음의 설명이 의미하는 용어를 쓰시오. 20-2

A는 한국IT 보안관제실에서 근무하게 되었다. A는 서비스 운용 중 외부 공격으로 인한 서버다운, 자연재해, 시스템 장애 등의 비상 상황에도 고객 응대 서비스를 정상적으로 수행하기 위해 구축한 시스템을 관리하는 업무를 수행한다. 이 용어는 위와 같은 비상 상황이 발생한 경우 "비상사태 또는 업무중단 시점부터 업무가 복구되어 다시 정상 가동 될 때까지의 시간"을 의미한다.

답: **RTO(목표복구시간)**

<RTO, RPO, RCO, RSO의 개념 비교>

구분	설명
RTO	Recovery Time Objectives: 재해복구 시간 목표(업무관점) 업무 중단 시점부터 복구되어 가동될 때 까지 시간 목표 예: 2시간, 7일, 1개월 구축 비용에 반비례하며, 재해 발생 손실에 비례
RPO	Recovery Point Objective: 재해복구 시점목표(데이터 관점) 재해 발생시 데이터손실을 수용 손실 허용 시점 예: 특정백업시점 데이터 복구, 전일 마감 백업 시점 복구
RCO	Recovery Communication Objective: 네트워크복구시간목표 주 영업점과 DR센터 간 네트워크 복구 수준 및 범위 예: 주요지점, 전 영업점 정보시스템 사용
RSO	Recovery Scope Objective: 재해복구 범위 목표 업무 중요도에 따른 복구 대상시스템 선정 예: 계정계, 정보계, 대외계 등

29. 데이터베이스 보안에 관련된 다음 설명에 해당하는 용어를 쓰시오. 21-1

접근통제는 데이터가 저장된 객체와 이를 사용하려는 주체 사이의 정보 흐름을 제한하는 것이다. 이러한 접근통제에 관한 기술 중 ()는 데이터에 접근하는 사용자의 신원에 따라 접근 권한을 부여하여 제어하는 방식으로, 데이터의 소유자가 접근통제 권한을 지정하고 제어한다. 객체를 생성한 사용자가 생성된 객체에 대한 모든 권한을 부여받고, 부여된 권한을 다른 사용자에게 허가할 수도 있다.

답: **DAC(임의 접근통제)**

32. 통합 구현과 관련하여 다음 설명의 괄호에 공통으로 들어갈 알맞은 용어를 쓰시오. 20-2

()는 HTTP, HTTPS, SMTP 등을 사용하여 xml 기반의 메시지를 네트워크 상에서 교환하는 프로토콜로, () envelope, 헤더(header), 바디(body) 등이 추가된 xml 문서이다. ()는 복잡하고 무거운 구조로 구성되어 있어 () 보다는 restful 프로토콜을 이용하기도 한다.

답: **SOAP(Simple Object Access Protocol)**

33. 웹 서비스(Web Service)와 관련된 다음 설명에 해당하는 용어를 쓰시오. 21-1

웹 서비스와 관련된 서식이나 프로토콜 등을 표준적인 방법으로 기술하고 게시하기 위한 언어로, XML로 작성되며 UDDI의 기초가 된다. SOAP, XML 스키마와 결합하여 인터넷에서 웹 서비스를 제공하기 위해 사용되며, 클라이언트는 이것을 통해 서버에서 어떠한 조작이 가능한지를 파악할 수 있다.

답: **WSDL(Web Services Description Language)**

[해설]

WSDL(Web Services Description Language)은 XML 기반의 인터페이스 정의 언어로, 웹 서비스가 제공하는 기능, 매개변수, 데이터 구조, 그리고 통신 프로토콜 및 주소(엔드포인트)를 기계가 읽을 수 있는 형식으로 기술하는 문서입니다.

37. 결합도(Coupling)

- 결합도는 모듈 간에 상호 의존하는 정도 또는 두 모듈 사이의 연관 관계
- 결합도가 약할수록 품질이 높고, 강할수록 품질이 낮음
- 결합도의 종류와 강도

내용 결합도 공통 결합도 외부 결합도 제어 결합도 스탬프 결합도 자료 결합도

종류	내용
내용 결합도 (Content Coupling)	한 모듈이 다른 모듈의 내부 기능 및 그 내부 자료를 직접 참조하거나 수정할 때의 결합도
공통(공유) 결합도 (Common Coupling)	공유되는 공통 데이터 영역을 여러 모듈이 사용할 때의 결합도
외부 결합도 (External Coupling)	어떤 모듈에서 선언한 데이터(변수)를 외부의 다른 모듈에서 참조할 때의 결합도
제어 결합도 (Control Coupling)	- 어떤 모듈이 다른 모듈 내부의 논리적인 흐름을 제어하기 위해 제어 신호나 제어 요소를 전달하는 결합도 P. 169로 - 하위 모듈에서 상위 모듈로 제어 신호가 이동하여 하위 모듈이 상위 모듈에게 처리 명령을 내리는 권리 전도 현상이 발생하게 됨
스탬프(검인) 결합도 (Stamp Coupling)	모듈 간의 인터페이스로 배열이나 레코드 등의 자료 구조가 전달될 때의 결합도 P. 200로
자료 결합도 (Data Coupling)	모듈 간의 인터페이스가 자료 요소로만 구성될 때의 결합도

◎ 응집도(Cohesion)

- 응집도는 모듈의 내부 요소들이 서로 관련되어 있는 정도
- 응집도가 강할수록 품질이 높고, 약할수록 품질이 낮음
- 응집도의 종류와 강도 [P. 225로](#)

기능적 응집도	순차적 응집도	교환적 응집도	절차적 응집도	시간적 응집도	논리적 응집도	우연적 응집도
---------	---------	---------	---------	---------	---------	---------

응집도 강함 ← → 응집도 약함

종류	내용 P. 166로
기능적 응집도 (Functional Cohesion)	모듈 내부의 모든 기능 요소들이 단일 문제와 연관되어 수행될 경우의 응집도
순차적 응집도 (Sequential Cohesion)	모듈 내 하나의 활동으로부터 나온 출력 데이터를 그 다음 활동의 입력 데이터로 사용할 경우의 응집도
교환(통신)적 응집도 (Communication Cohesion)	동일한 입력과 출력을 사용하여 서로 다른 기능을 수행하는 구성 요소들이 모였을 경우의 응집도
절차적 응집도 (Procedural Cohesion)	모듈이 다수의 관련 기능을 가질 때 모듈 안의 구성 요소들이 그 기능을 순차적으로 수행할 경우의 응집도
시간적 응집도 (Temporal Cohesion)	특정 시간에 처리되는 몇 개의 기능을 모아 하나의 모듈로 작성할 경우의 응집도
논리적 응집도 (Logical Cohesion)	유사한 성격을 갖거나 특정 형태로 분류되는 처리 요소들로 하나의 모듈이 형성되는 경우의 응집도
우연적 응집도 (Coincidental Cohesion)	모듈 내부의 각 구성 요소들이 서로 관련 없는 요소로만 구성된 경우의 응집도

41. 네트워크에 관련된 다음 설명에 해당하는 용어를 쓰시오. [21-1](#)

모듈 간 통신 방식을 구현하기 위해 사용되는 대표적인 프로그래밍 인터페이스 집합으로, 복수의 프로세스를 수행하며 이뤄지는 프로세스 간 통신까지 구현이 가능하다. 대표적인 메소드에는 공유 메모리(Shared Memory), 소켓(Socket), 세마포어(Semaphores), 파이프와 네임드 파이프(Pipes&named Pipes), 메시지 큐잉(Message Queueing)이 있다.

답: **IPC(Inter-Process Communication)**

49. 클라이언트와 서버 간 자바스크립트 및 XML을 비동기 방식으로 처리하며, 전체 페이지를 새로 고치지 않고도 웹페이지 일부 영역만을 업데이트할 수 있도록 하는 기술을 의미하는 용어를 쓰시오. [20-2, 23-1, 25-2](#)

답: **AJAX(Asynchronous JavaScript and XML)**

57. 애플리케이션 테스트에서 사용되는 살충제 패러독스(Pesticide Paradox)의 개념을 간략히 설명하시오. [20-1](#)

답: **살충제 패러독스는 동일한 테스트 케이스로 동일한 테스트를 반복하면 더 이상 결함이 발견되지 않는 현상을 의미한다.**

73. 애플리케이션 성능이란 사용자가 요구한 기능을 최소한의 자원을 사용하여 최대한 많은 기능을 신속하게 처리하는 정도를 나타낸다. 애플리케이션 성능 측정의 지표에 대한 다음 설명에서 괄호(①~③)에 들어갈 알맞은 용어를 쓰시오. [20-1](#)

(①)	일정 시간 내에 애플리케이션이 처리하는 일의 양을 의미한다.
(②)	애플리케이션에 요청을 전달한 시간부터 응답이 도착할 때까지 걸린 시간을 의미한다.
(③)	애플리케이션에 작업을 의뢰한 시간부터 처리가 완료될 때까지 걸린 시간을 의미한다.
자원 활용률	애플리케이션이 의뢰한 작업을 처리하는 동안의 CPU, 메모리, 네트워크 등의 자원 사용률을 의미한다.

답: ① **처리량(Throughput)**
② **응답 시간(Response Time)**
③ **경과 시간(Turn Around Time)**

76. 데이터를 제어하는 DCL의 하나인 ROLLBACK에 서술하시오. [20-2](#)

답: **ROLLBACK은 트랜잭션이 실패한 경우 작업을 취소하고 이전 상태로 되돌리기 위한 명령이다.**

78. 보안 위협의 하나인 SQL Injection에 서술 [20-2](#)

답: **SQL Injection은 웹 응용 프로그램에 SQL 구문을 삽입하여 내부 데이터베이스(DB) 서버의 데이터를 유출 및 변조하고 관리자 인증을 우회하는 공격 기법이다.**

79. 암호화 알고리즘에 대한 다음 설명에서 괄호(①, ②)에 들어갈 알맞은 용어를 쓰시오. [22-2](#)

- 암호화 알고리즘은 패스워드, 주민번호, 은행계좌와 같은 중요 정보를 보호하기 위해 평문을 암호화된 문장으로 만드는 절차 또는 방법을 의미한다.
- 스위스의 라이(Lai)와 메시(Massey)는 1990년 PES를 발표하고, 이후 이를 개선한 IPES를 발표하였다. IPES는 128비트의 Key를 사용하여 64비트 블록을 암호화하는 알고리즘이며 현재는 (①)라고 불린다.

- (②)은 국가 안전 보장국(NSA)에서 개발한 암호화 알고리즘으로, 클리퍼 칩(Clipper Chip)이라는 IC 칩에 내장되어 있다. 80비트의 Key를 사용하여 64비트 블록을 암호화하며, 주로 전화기와 같은 음성 통신 장비에 삽입되어 음성 데이터를 암호화한다.

답: ① **IDEA(International Data Encryption Algorithm)**
② **Skipjack(스킵잭)**

84. 보안 위협에 관한 다음 설명에서 괄호에 공통으로 들어갈 알맞은 용어를 쓰시오. [21-3](#)

() 스푸핑은 로컬 네트워크(LAN)에서 사용하는 () 프로토콜의 취약점을 이용한 공격 기법으로, 자신의 물리적 주소(MAC)를 변조하여 다른 PC에게 도달해야 하는 데이터 패킷을 가로채거나 방해한다.

답: **ARP(Address Resolution Protocol)**

[해설]주소 결정 프로토콜(Address Resolution Protocol)

92. 헝가리안 표기법(Hungarian Notation)에 대해 간략히 서술하시오. [20-3](#)

답: 헝가리안 표기법은 변수명 작성시 변수의 자료형을 알 수 있도록 자료형을 의미하는 문자를 포함하여 작성하는 방법이다.

93. 스니핑(Sniffing)은 사전적 의미로 '코를 킁킁 거리다, 냄새를 맡다'이다. 네트워크 보안에서 스니핑에 대한 개념을 간략히 한 문장(1 문장)으로 쓰시오. [20-4](#)

답: 스니핑은 네트워크의 중간에서 남의 패킷 정보를 도청하는 해킹 유형의 하나로 수동적 공격에 해당한다.

94. C++에서 생성자(Constructor)에 대해 간략히 서술하시오. [20-3](#)

답: 생성자는 객체 변수 생성에 사용되는 메소드로, 객체 변수를 생성하면서 초기화를 수행한다.

95. 다음 설명에 해당하는 운영체제(OS)를 쓰시오. [20-4](#)

- 1960년대 AT&T 벨(Bell) 연구소가 MIT, General Electric사와 함께 공동 개발한 운영체제이다.
- 시분할 시스템(Time Sharing System)을 위해 설계된 대화식 운영체제이다.
- 대부분 C 언어로 작성되어 있어 이식성이 높으며 장치, 프로세스 간의 호환성이 높다.
- 트리 구조의 파일 시스템을 갖는다.

답: **UNIX(유닉스)**

113. 다음 네트워크 관련 설명에서 괄호에 들어갈 알맞은 용어를 영문(Full name 또는 약어)으로 쓰시오. [20-3, 23-1](#)

IP(Internet Protocol)의 주요 구성원 중 하나로, OSI 계층 모델의 네트워크 계층에 속한다. 네트워크 컴퓨터의 운영체제에서 오류 메시지를 수신하거나, 전송 경로를 변경하는 등 오류 처리를 위한 제어 메시지를 주로 취급한다. 관련된 도구로 traceroute, ping이 있으며, ping of death와 같은 네트워크 공격 기법에 활용되기도 한다.

()는 TCP/IP 기반의 인터넷 통신 서비스에서 인터넷 프로토콜(IP)과 조합하여 통신 중에 발생하는 오류의 처리와 전송 경로의 변경 등을 위한 제어 메시지를 취급하는 무연결 전송용 프로토콜로, OSI 기본 참조 모델의 네트워크 계층에 속한다.

답: **ICMP(Internet Control Message Protocol)**

123. 분산 컴퓨팅에 대한 다음 설명에 해당하는 용어를 쓰시오. [20-4](#)

- 오픈 소스 기반 분산 컴퓨팅 플랫폼이다.
- 분산 저장된 데이터들은 클러스터 환경에서 병렬 처리된다.
- 일반 PC급 컴퓨터들로 가상화된 대형 스토리지를 형성하고 그 안에 보관된 거대한 데이터 세트를 병렬로 처리할 수 있도록 개발되었다.
- 더그 커팅과 마이크 캐퍼렐라가 개발했으며, 구글의 맵리듀스(MapReduce) 엔진을 사용하고 있다.

답: **하둡(Hadoop)**

124. 데이터 마이닝(Data Mining)의 개념을 간략히 서술하시오. [20-1](#)

답: 데이터 마이닝은 대량의 데이터를 분석하여 데이터에 내재된 변수 사이의 상호관계를 규명하여 일정한 패턴을 찾아내는 기법이다.

126. 데이터베이스의 병행제어(Concurrency Control) 기법 중 하나로, 접근한 데이터에 대한 연산을 모두 마칠 때까지 추가적인 접근을 제한함으로써 상호 배타적으로 접근하여 작업을 수행하도록 하는 기법을 쓰시오. [21-2](#)

답: **로킹(Locking)**

129. 소프트웨어 개발에서의 작업 중 형상 통제에 대해 간략히 서술하시오. [20-3](#)

답: 형상 통제는 식별된 형상 항목에 대한 변경 요구를 검토하여 현재의 기준선이 잘 반영될 수 있도록 조정하는 작업이다.

142. 서비스 거부(DoS) 공격 기법 중 TCP가 신뢰성 있는 전송을 위해 사용하는 3-way-handshake 과정을 의도적으로 중단시킴으로써 공격 대상지인 서버가 대기 상태에 놓여 정상적인 서비스를 수행하지 못하게 하는 공격 방법을 가리키는 용어를 쓰시오. [25-2](#)

답: **SYN Flooding**

[해설]

TCP 3-Way Handshake 취약점을 이용한 네트워크 공격 분석

- **SYN Flooding**은 정상적인 연결 생성 과정에서 서버가 Client로부터 SYN 패킷을 받은 후 응답(YN+ACK)을 보내고, 마지막 ACK를 기다리는 대기 상태(Half-Open)의 자원 제한을 악용하는 대표적인 **DoS(서비스 거부)** 공격입니다.
- 공격자는 가상의 IP 주소로 다량의 SYN 패킷을 보내 서버의 연결 대기 큐(Queue)를 꽉 채우며, 이로 인해 일반 사용자가 서버에 접속하지 못하게 만듭니다.
- **세션 하이재킹(Session Hijacking)** [145로](#)

상호 인증 과정을 거친후 접속해 있는 서버와 서로 접속되어 클라이언트 사이의 세션 정보를 가로챈 후 인증 정보 없이도 가로챈 세션을 이용해 공격자가 원래의 클라이언트인 것처럼 위장하여 서버의 자원이나 데이터를 무단으로 사용하는것으로, TCP 3-Way-Handshake 과정에 끼어들으로써 클라이언트와 서버 간의 동기화된 시퀀스 번호를 가로채 서버에 무단으로 접근함.

공격 기법	핵심 원리 (TCP 연결 관점)	주요 공격 목적
SYN Flooding	3-Way Handshake 과정을 의도적으로 중단/지연 시켜 서버를 대기 상태로 만듦	서버 자원 고갈 (서비스 마비)
Session Hijacking	연결 확립 후, 클라이언트와 서버 간의 동기화된 시퀀스 번호 를 가로채어 사이에 끼어들	인증 우회 및 데이터/자원 무단 사용

◎ 서비스 거부(DoS; Denial of Service) 공격

서비스 거부 공격이란 표적이 되는 서버의 자원을 고갈시킬 목적으로 다수의 공격자 또는 시스템에서 **대량의 데이터를 한 곳의 서버에 집중적으로 전송함으로써**, 표적이되는 **서버의 정상적인 기능을 방해하는 것**

주요 서비스 거부 공격의 유형

1. Ping of Death(죽음의 핑)

- Ping of Death는 Ping 명령을 전송할 때 **패킷의 크기를 인터넷 프로토콜 허용 범위 이상으로 전송하여** 공격 대상의 **네트워크를 마비시키는 서비스 거부 공격 방법**
- 공격에 사용되는 큰 패킷은 수백 개의 패킷으로 분할되어 전송되는데, 공격 대상은 분할된 대량의 패킷을 수신함으로써 분할되어 전송된 패킷을 재조립 해야 하는 부담과 분할되어 전송된 각각의 패킷들의 **ICMP Ping** 메시지에 대한 응답을 처리 하느라 시스템이 다운되게됨

2. SMURFING(스머핑)

- SMURFING은 **IP나 ICMP의 특성을 악용하여 엄청난 양의 데이터를 한 사이트에 집중적으로 보냄으로써 네트워크를 불능 상태로 만드는 공격 방법**
- 공격자는 송신 주소를 공격 대상지의 IP 주소로 위장하고 해당 네트워크 라우터의 **브로드캐스트 주소**를 수신지로 하여 패킷을 전송하면, 라우터의 브로드캐스트 주소로 수신된 패킷은 해당 네트워크 내의 모든 컴퓨터로 전송됨
- 해당 네트워크 내의 모든 컴퓨터는 수신된 패킷에 대한 응답 메시지를 송신 주소인 공격 대상지로 집중적으로 전송하게 되는데, 이로 인해 공격 대상지는 네트워크 과부하로 인해 정상적인 서비스를 수행할 수 없게 됨
- SMURFING 공격을 무력화하는 방법 중 하나는 각 네트워크 라우터에서 **브로드캐스트 주소를 사용할 수 없게 미리 설정해** 놓는 것

3. SYN Flooding

- TCP(Transmission Control Protocol)는 신뢰성 있는 전송을 위해 3-way-handshake를 거친 후에 데이터를 전송하게 되는데, **SYN Flooding**은 공격자가 가상의 클라이언트로 위장하여 **3-way-handshake** 과정을 의도적으로 중단시킴으로써 공격 대상지인 서버가 대기 상태에 놓여 정상적인 서비스를 수행하지 못하게 하는 공격 방법
- SYN Flooding에 대비하기 위해 수신지의 'SYN' 수신 대기 시간을 줄이거나 침입 차단 시스템을 활용함

4. TearDrop

- 데이터의 송수신 과정에서 패킷의 크기가 커 여러 개로 분할되어 전송될 때 분할 순서를 알 수 있도록 Fragment Offset 값을 함께 전송하는데, **TearDrop**은 이 **Offset** 값을 변경시켜 수신 측에서 패킷을 재조립할 때 오류로 인한 과부하를 발생시킴으로써 시스템이 다운되도록 하는 공격 방법
- TearDrop에 대비하기 위해 Fragment Offset이 잘못된 경우 해당 패킷을 폐기하도록 설정함

5. LAND Attack(Local Area Network Denial Attack) [▶85로](#)

- LAND Attack은 패킷을 전송할 때 **송신 IP 주소와 수신 IP 주소를 모두 공격 대상의 IP 주소**로 하여 공격 대상에게 전송하는 것으로, 이 패킷을 받은 공격 대상은 송신 IP 주소가 자신이므로 자신에게 응답을 수행하게 되는데, 이러한 패킷이 계속해서 전송될 경우 **자신에 대해 무한히 응답하게 하는 공격**
- LAND Attack에 대비하기 위해 송신 IP 주소와 수신 IP 주소의 적절성을 검사함

6. DDoS 공격(Distributed Denial of Service, 분산 서비스 거부) 공격

- DDoS 공격은 **여러 곳에 분산된 공격 지점에서 한 곳의 서버에 대해 분산 서비스 공격을 수행하는 것**
- 네트워크에서 취약점이 있는 호스트들을 탐색한 후 이들 호스트들에 분산 서비스 공격용 톨을 설치하 에이전트(Agent)로 만든 후 DDoS 공격에 이용함
- **분산 서비스 공격용 톨**: 에이전트(Agent)의 역할을 수행하도록 설계된 프로그램으로 데몬(Daemon)이라고 부르며, 다음과 같은 종류가 있음

155. 다음 설명에 해당하는 알맞은 용어를 쓰시오. [24-3](#)

- IP나 ICMP의 특성을 악용하여 엄청난 양의 데이터를 한 사이트에 집중적으로 보냄으로써 네트워크를 불능 상태로 만드는 공격 방법이다.
- 공격자는 송신 주소를 공격 대상지의 IP 주소로 위장하고 해당 네트워크 라우터의 브로드캐스트 주소를 수신지로 하여 패킷을 전송하면, 라우터의 브로드캐스트 주소로 수신된 패킷은 해당 네트워크 내의 모든 컴퓨터로 전송된다.
- 해당 네트워크 내의 모든 컴퓨터는 수신된 패킷에 대한 응답 메시지를 송신 주소인 공격 대상지로 집중적으로 전송하게 되는데, 이로 인해 공격 대상지는 네트워크 과부하로 인해 정상적인 서비스를 수행할 수 없게 된다.

답: 스머핑 (Smurfing 또는 Smurf Attack)

171. 다음 설명에 해당하는 라우팅 프로토콜을 쓰시오. [24-1](#)

- RIP의 단점을 해결하여 새로운 기능을 지원하는 인터넷 프로토콜이다.
- 최단 경로 탐색에 Dijkstra 알고리즘을 사용한다.
- 대규모 네트워크에서 많이 사용된다.
- 링크 상태를 실시간으로 반영하여 최단 경로로 라우팅을 지원한다.

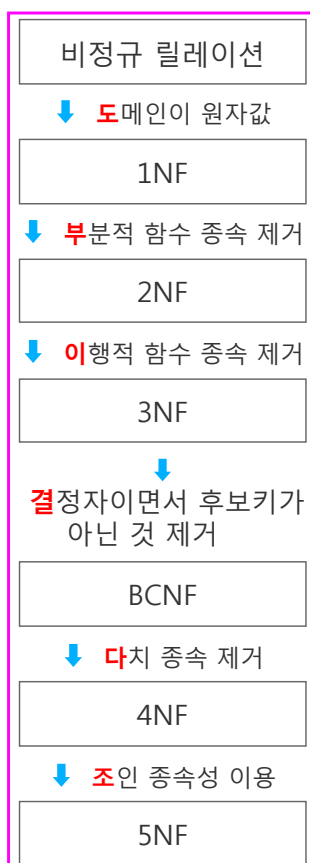
답: OSPF

[해설]

링크 상태 라우팅 프로토콜 - OSPF (Open Shortest Path First)

- **링크 상태(Link State) 알고리즘**: 거리 벡터 방식인 RIP와 달리 네트워크 토폴로지 변경 사항을 실시간으로 링크 상태 광고(LSA)를 통해 전파하고 전 지도의 맵을 구성합니다.
- **다익스트라(Dijkstra) 알고리즘 활용**: 모든 라우터가 최단 경로 트리(Shortest Path Tree)를 독자적으로 계산하여 경로 비용(Cost) 메트릭이 최소인 최적의 루트를 산출합니다.
- **대규모 네트워크 적합성**: 홉 카운트 제약(최대 15홉)이 있는 RIP와 달리 Area 단위를 나누어 대규모망을 체계적이고 효율적으로 관리할 수 있습니다.

173. 정규화(Normalization) 과정



185. 소프트웨어 데이터의 비정상적인 수정이 감지되면 소프트웨어를 오작동하게 만들어 악용을 방지하는 기술이다. 해시 함수, 핑거 프린트, 워터마킹 등의 보안 요소를 생성하여 소프트웨어에 삽입하고, 실행코드를 난독화하며, 실행 시 원본 비교 및 데이터 확인을 수행함으로써 소프트웨어를 보호하는 이 기술을 가리키는 용어를 쓰시오. [23-2](#)

답: **템퍼 프루핑 (Tamper Proofing / 역변조 방지)**

[해설]

템퍼 프루핑 (Tamper Proofing) 보안 기술 분석

- 프로그램이나 데이터의 **비정상적인 위·변조(Tampering)** 행위가 감지되었을 때, 소프트웨어 스스로 작동을 멈추거나 의도적인 오작동(장애)을 유발하여 크래킹 및 무단 악용을 원천 차단하는 능동형 보안 방어 기술입니다.
- 지문에 제시된 것처럼 무결성을 검증하기 위한 **해시 함수**, 저작권 추적을 위한 **핑거프린팅 및 워터마킹**, 리버스 엔지니어링을 어렵게 만드는 **코드 난독화(Obfuscation)** 기술 등이 유기적으로 결합되어 시스템의 견고함을 높입니다.

3. 다음은 <제품>(제품명, 단가, 제조사) 테이블을 대상으로 "H" 제조사에서 생산한 제품들의 '단가'보다 높은 '단가'를 가진 제품의 정보를 조회하는 <SQL문>이다. 괄호에 알맞은 답을 적어 <SQL문>을 완성하시오. [22-2](#)

```
SELECT 제품명, 단가, 제조사
FROM 제품
WHERE 단가 > ( ) (SELECT 단가 FROM 제품 WHERE 제조사 = 'H');
```

답: **ALL**

8. <학생> 테이블에서 '이름'이 "민수"인 튜플을 삭제하고자 한다. 다음 <처리 조건>을 참고하여 SQL문을 작성하시오. [20-3, 23-1](#)

<처리 조건>

- 최소한의 코드로 작성될 수 있도록 SQL문을 구성한다.
- 명령문 마지막의 세미콜론(;)은 생략이 가능하다.
- 인용 부호가 필요한 경우 작은따옴표(' ')를 사용한다.

답: **DELETE FROM 학생 WHERE 이름 = '민수';**

9. 다음 <속성 정의서>를 참고하여 <학생> 테이블에 대해 20자의 가변 길이를 가진 '주소' 속성을 추가하는 <SQL문>을 완성하시오. [20-3](#)

<속성 정의서>

속성명	데이터타입	제약조건	테이블명
학번	CHAR(10)	UNIQUE	학생
이름	VARCHAR(8)	NOT NULL	학생
주민번호	CHAR(13)		학생
학과	VARCHAR(16)	FOREIGN KEY	학생
학년	INT		학생

<SQL문>

```
( ① ) TABLE 학생 ( ② ) 주소 VARCHAR(20);
```

답: ① **ALTER** ② **ADD**

10. 다음 <학생> 테이블을 참고하여 <처리 조건>에서 요구하는 SQL문을 작성하시오. [20-2](#)

<학생>

학번 (varchar)	이름 (varchar)	학년 (number)	수강과목 (varchar)	점수 (number)	연락처 (varchar)
20E0232	김인영	3	세무행정	4.5	010-5412-4544
19D0024	이성화	2	토목개론	3	010-1548-4796
20E0135	성유수	4	실용법학	3.5	010-9945-7411
20E0511	우인혁	1	데이터론	2	010-3451-4972

<처리 조건>

- 3, 4학년의 학번, 이름을 조회한다.
- IN 예약어를 사용해야 한다.
- 속성명 아래의 괄호는 속성의 자료형을 의미한다.

답: **SELECT 학번, 이름 FROM 학생 WHERE 학년 IN (3, 4);**

11. 다음 <student> 테이블을 참고하여 'name' 속성으로 'idx_name'이라는 인덱스를 생성하는 SQL문을 작성하시오. [20-2](#)

<student>

stid	name	score	deptid
2001	brown	85	PE01
2002	white	45	EF03
2003	black	67	UW11

답: **CREATE INDEX idx_name ON student(name);**

[정렬]

12. 다음은 <성적> 테이블에서 이름(name)과 점수(score)를 조회하되, 점수를 기준으로 내림차순 정렬하여 조회하는 <SQL문>이다. 괄호(①~③)에 알맞은 답을 적어 <SQL문>을 완성하시오. [22-1](#)

<성적>

name	class	score
정기찬	A	85
이영호	C	74
황정형	C	95
김지수	A	90
최은영	B	82

<SQL문>

```
SELECT name, score
FROM 성적
( ① ) BY ( ② ) ( ③ );
```

답: ① **ORDER** ② **score** ③ **DESC**

14. 다음 질의 내용에 대한 SQL문을 완성하시오. [20-4](#)

질의	학생 테이블에서 학과별 튜플의 개수를 검색하시오. (단, 아래의 실행 결과가 되도록 한다.)
----	--

<학생>

학번	이름	학년	학과	주소
20160011	김영란	2	전기	서울
19210113	이재우	3	컴퓨터	대구
21168007	함소진	1	전자	부산
19168002	우혜정	3	전자	광주
18120073	김진수	4	컴퓨터	울산

<실행결과>

학과	학과별튜플수
전기	1
컴퓨터	2
전자	2

<처리 조건>

- WHERE 조건절은 사용할 수 없다.
- GROUP BY는 반드시 포함한다.
- 집계함수(Aggregation Function)를 적용한다.
- 학과별튜플수 컬럼이름 출력에 Alias(AS)를 활용한다.
- 문장 끝의 세미콜론(;)은 생략해도 무방하다.
- 인용부호 사용이 필요한 경우 단일 따옴표(' ')를 사용한다.

답: **SELECT 학과, COUNT(*) AS 학과별튜플수 FROM 학생 GROUP BY 학과;**

15. 다음 <성적> 테이블에서 과목별 점수의 평균이 90점 이상인 '과목이름', '최소점수', '최대점수'를 검색하고자 한다. <처리조건>을 참고하여 적합한 SQL문을 작성하시오. [20-3](#), [23-1](#)

<성적>

학번	과목번호	과목이름	학점	점수
a2001	101	컴퓨터구조	6	95
a2002	101	컴퓨터구조	6	84
a2003	302	데이터베이스	5	89
a2004	201	인공지능	5	92
a2005	302	데이터베이스	5	100
a2006	302	데이터베이스	5	88
a2007	201	인공지능	5	93

<결과>

과목이름	최소점수	최대점수
데이터베이스	88	100
인공지능	92	93

<처리조건>

- 최소한의 코드로 작성될 수 있도록 SQL문을 구성한다.
- WHERE문은 사용하지 않는다.
- GROUP BY와 HAVING을 이용한다.
- 집계함수(Aggregation Function)를 사용하여 명령문을 구성한다.
- '최소점수', '최대점수'는 별칭(Alias)을 위한 AS문을 이용한다.
- 명령문 마지막의 세미콜론(;)은 생략이 가능하다.
- 인용 부호가 필요한 경우 작은따옴표(' ')를 사용한다.

답: **SELECT 과목이름, MIN(점수) AS 최소점수, MAX(점수) AS 최대점수 FROM 성적 GROUP BY 과목이름 HAVING AVG(점수) >= 90;**

17. 다음 <사원> 테이블과 <동아리> 테이블을 조인(Join)한 <결과>를 확인하여 <SQL문>의 괄호(①, ②)에 들어갈 알맞은 답을 쓰시오. [21-2](#)

<사원>

코드	이름	부서
1601	김명해	인사
1602	이진성	경영지원
1731	박영광	개발
2001	이수진	-

<동아리>

코드	동아리명
1601	테니스
1731	탁구
2001	볼링

⇒

<결과>

코드	이름	동아리명
1601	김명해	테니스
1602	이진성	NULL
1731	박영광	탁구
2001	이수진	볼링

<SQL문>

SELECT a.코드, 이름, 동아리명 FROM 사원 a LEFT JOIN 동아리 b (①) a.코드 = b.(②);

답: ① ON ② 코드

21. 다음에 제시된 <학생>과 <학부> 테이블에 대한 릴레이션 스키마를 참고하여 각 SQL문의 요구사항에 맞도록 괄호(①~④)에 알맞은 답을 적어 SQL문을 완성하시오. [24-2](#)

릴레이션 스키마

<학생>

학번(PK)	이름	나이	학과
--------	----	----	----

<학부>

학번(PK)	이름	주소	나이
--------	----	----	----

<SQL 2> 문

[요구사항] <SQL 1> 문에서 추가한 학생을 <학부> 테이블에서 검색한 후 검색된 자료를 <학생> 테이블에 추가한다.
[SQL] INSERT INTO 학생(학번, 이름, 나이, 학과) (②) 학번, 이름, 나이, '컴퓨터공학' FROM 학부 WHERE 이름 = '최재균';

<SQL 3> 문

[요구사항] <학생> 테이블의 모든 자료를 조회한다.
[SQL] SELECT * (③) 학생;

답: ② SELECT ③ FROM

25. 다음 <학생> 테이블에 (9816021, '한국산', 3, '경영학개론', '050-1234-1234')인 데이터를 삽입하고자 한다. <처리조건>을 참고하여 적합한 SQL문을 작성하시오. [23-2](#)

<학생>

학번	이름	학년	신청과목	연락처
9815932	김태산	3	경영정보시스템	050-5234-1894
9914511	박명록	2	경제학개론	050-1415-4986
0014652	이익명	1	국제경영	050-6841-6781
9916425	김혜리	2	재무관리	050-4811-1187
9815945	이지영	3	인적자원관리	050-9785-8845

<처리조건>

- 최소한의 코드로 작성될 수 있도록 SQL문을 구성한다.
- 명령문 마지막의 세미콜론(;)은 생략이 가능하다.
- 인용 부호가 필요한 경우 작은따옴표('')를 사용한다.

답: INSERT INTO 학생 VALUES (9816021, '한국산', 3, '경영학개론', '050-1234-1234');

[해설]

- SELECT(검색): SELECT~ FROM~ WHERE~
- INSERT(삽입): INSERT INTO~ VALUES~
- DELETE(삭제): DELETE~ FROM~ WHERE~
- UPDATE(변경): UPDATE~ SET~ WHERE~

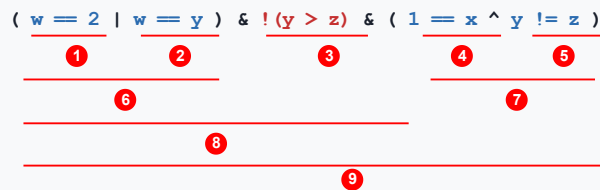
프로그래밍 - JAVA 8문제 001로

1. 다음 JAVA로 구현된 프로그램을 분석하여 그 실행 결과를 쓰시오. 21-3

```
public class Test {
    public static void main(String[] args) {
        int w = 3, x = 4, y = 3, z = 5;
        if ((w == 2 | w == y) & !(y > z) & (1 == x ^ y != z)) {
            w = x + y;
            if (7 == x ^ y != w)
                System.out.println(w);
            else
                System.out.println(x);
        }
        else {
            w = y + z;
            if (7 == y ^ z != w)
                System.out.println(w);
            else
                System.out.println(z);
        }
    }
}
```

답: 7

[해설]



- ① $w(3) \neq 2$ 거짓(0) ② $w(3) = y(3)$ 참(1)이며, ⑥ ① | ② (비트 or) 연산하면 1이다.
- ③ $y(3) > z(5)$ 거짓(0)이지만, 앞에 !(논리 not)가 있으므로 참(1)이다.
- ④ $1 \neq x(4)$ 거짓(0) ⑤ $y(3) \neq z(5)$ 참(1)이다.

• ⑥

$$\begin{array}{r} 0000(0) \\ | 0001(1) \\ \hline 0001(1) \end{array}$$

- ⑦ ④ ^ ⑤ : ④의 결과 0과 ⑤의 결과 1을 ^ (비트 xor) 연산하면 결과는 1이다.

$$\begin{array}{r} 0000(0) \\ ^ 0001(1) \\ \hline 0001(1) \end{array}$$

$$\begin{array}{r} 0001(1) \\ \& 0001(1) \\ \hline \end{array}$$

• ⑧ ⑥ & ③ : ⑥의 결과 1과 ③의 결과 1을 &(비트 and) 연산하면 결과는 1이다.

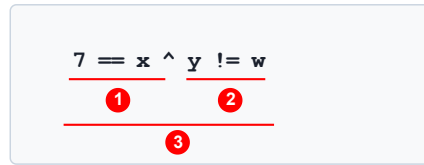
0001(1)

• ⑨ ⑧ & ⑦ : ⑧의 결과 1과 ⑦의 결과 1을 &(비트 and) 연산하면 결과는 1이다.

∴ 최종 결과는 1이며, 1은 조건에서 참(True)을 의미하므로 ③번으로 이동한다.

③ w에 x와 y의 합을 저장한다. (w = 7)

④ 조건이 참이면 ⑤번 문장을, 거짓이면 ⑤번 아래 else 다음 문장을 수행한다. 연산자 우선순위에 따라 다음의 순서로 조건의 참/거짓을 확인한다.



• ① : 7과 x의 값 4는 같지 않으므로 결과는 거짓(0)이다.

• ② : y의 값 3과 w의 값 7은 같지 않으므로 결과는 참(1)이다.

• ③ ① ^ ② : ①의 결과 0과 ②의 결과 1을 ^ (비트 xor) 연산하면 결과는 1이다.

∴ 최종 결과는 1이며, 1은 조건에서 참을 의미하므로 ⑤번 문장을 수행한다.

⑤ w의 값 7을 출력한다.

6. 다음 Java로 구현된 프로그램을 분석하여 그 실행 결과를 쓰시오. [20-3](#)

```
public class Test {  
    public static void main(String[] args) {  
        int a = 0, sum = 0;  
        while (a < 10) {  
            a++;  
            if (a % 2 == 1)  
                continue;  
            sum += a;  
        }  
        System.out.println(sum);  
    }  
}
```

답: 30

[해설]

2 4 6 8 10 = 30

4. 다음 C언어로 구현된 프로그램을 분석하여 그 실행 결과를 쓰시오. [21-2](#)

```
#include <stdio.h>  
main() {  
    int res = mp(2, 10);  
    printf("%d", res);  
}  
  
int mp(int base, int exp) {  
    int res = 1;  
    for (int i = 0; i < exp; i++)  
        res *= base;  
    return res;  
}
```

```
}
```

답: 1024

[해설]

2 4 6 8 16 32 64 128 256 512 1024(9)

12. 다음 C 언어로 구현된 프로그램을 분석하여 그 실행 결과를 쓰시오. ★ 25-3

```
#include <stdio.h>
struct Node {
    struct Node* next;
    unsigned int x;
};
int main( ) {
    struct Node t1 = { 0, 5u };
    struct Node t2 = { 0, 7u };
    struct Node t3 = { 0, 11u };
    t3.next = &t2;
    t2.next = &t1;
    struct Node* curr = &t3;
    int sum = 0;
    while (curr) {
        sum = sum * 3 + curr->x;
        curr = curr->next;
    }
    sum = (sum ^ 42u) + 100u;
    printf("%u\n", sum);
}
```

답: 187

[해설]

구조체 포인터 연결 리스트의 순회 및 비트 연산 프로세스 분석

• 연결 리스트 형성 구조 파악:

노드 노드가 연결된 방향을 보면 t3.next → t2, t2.next → t1, 그리고 t1.next = 0(NULL) 순서로 가리킵니다. 따라서 탐색 포인터 curr는 t3 → t2 → t1 순으로 총 3번 루프를 돌며 순회합니다.

• while 루프를 통한 누적 연산 (sum = sum * 3 + curr->x):

- 1회차 (t3 탐색): sum = 0 * 3 + 11 = 11

- 2회차 (t2 탐색): sum = 11 * 3 + 7 = 40

- 3회차 (t1 탐색): sum = 40 * 3 + 5 = 125

루프 종료 시점의 sum 최종 저장값은 125가 됩니다.

• 최종 비트 배타적 논리합(XOR) 및 덧셈 연산:

먼저 괄호 내부의 sum ^ 42u 연산을 수행합니다. 10진수 125와 42를 이진수로 변환하여 XOR 연산하면:

- 125 = 0111 1101₂

- 42 = 0010 1010₂

- XOR 결과 = 0101 0111₂ → 10진수로 복원하면 87이 됩니다.

마지막으로 87 + 100u를 연산하여 최종 변수값은 부호 없는 10진수 서식(%u)에 의해 187이 출력됩니다.

sum	*curr
0	3000
11	2000
40	1000
125	0

결과 187

13. 다음 C 언어로 구현된 프로그램을 분석하여 그 실행 결과를 쓰시오. [25-2](#)

```
#include <stdio.h>
#include <stdlib.h>
struct node {
    int p;
    struct node* n;
};
int main() {
    struct node a = {1, NULL};
    struct node b = {2, NULL};
    struct node c = {3, NULL};
    a.n = &b; b.n = &c; c.n = NULL;
    c.n = &a; a.n = &b; b.n = NULL;
    struct node* head = &c;
    printf("%d %d %d", head->p, head->n->p, head->n->n->p);
    return 0;
}
```

답: 3 1 2

17. 다음 C 언어로 구현된 프로그램을 분석하여 그 실행 결과를 쓰시오. [25-1](#)

```
#include <stdio.h>
char Data[5] = {'B', 'A', 'D', 'E'};
char c;

int main() {
    int i, temp, temp2;
    c = 'C';
    printf("%d\n", Data[3]-Data[1]);
    for(i = 0; i < 5; ++i) {
        if(Data[i] > c)
            break;
    }
    temp = Data[i];
    Data[i] = c;
    i++;
    for(; i < 5; ++i) {
        temp2 = Data[i];
        Data[i] = temp;
        temp = temp2;
    }

    for(i = 0; i < 5; i++) {
        printf("%c", Data[i]);
    }
}
```

답: 4
BACDE

18. 다음 C 언어로 구현된 프로그램을 분석하여 그 실행 결과를 쓰시오. [25-1](#)

```

#include <stdio.h>
#include <stdlib.h>
void set(int** arr, int* data, int rows, int cols) {
    for (int i = 0; i < rows * cols; ++i) {
        arr[((i + 1) / rows) % rows][(i + 1) % cols] = data[i];
    }
}
int main() {
    int rows = 3, cols = 3, sum = 0;
    int data[] = {5, 2, 7, 4, 1, 8, 3, 6, 9};
    int** arr;
    arr = (int**)malloc(sizeof(int*) * rows);
    for (int i = 0; i < rows; i++) {
        arr[i] = (int*)malloc(sizeof(int) * cols);
    }
    set(arr, data, rows, cols);
    for (int i = 0; i < rows * cols; i++) {
        sum += arr[i / rows][i % cols] * (i % 2 == 0 ? 1 : -1);
    }
    for (int i = 0; i < rows; i++) {
        free(arr[i]);
    }
    free(arr);
    printf("%d", sum);
}

```

답: 13

19. 다음 C 언어로 구현된 프로그램을 분석하여 그 실행 결과를 쓰시오. [25-1](#)

```

#include <stdio.h>
typedef struct student {
    char* name;
    int score[3];
} Student;
int dec(int enc) {
    return enc & 0xA5;
}
int sum(Student* p) {
    return dec(p->score[0]) + dec(p->score[1]) + dec(p->score[2]);
}
int main() {
    Student s[2] = { {"Kim", {0xA0, 0xA5, 0xDB}}, {"Lee", {0xA0, 0xED, 0x81}} };
    Student* p = s;
    int result = 0;
    for (int i = 0; i < 2; i++) {
        result += sum(&s[i]);
    }
    printf("%d", result);
    return 0;
}

```

답: 908

[해설]

학생 (i)	원래 16진수 배열	비트 AND 연산 (& 0xA5)	변환된 10진수 합
s[0] (Kim)	{0xA0, 0xA5, 0xDB}	0xA0, 0xA5, 0x81	160 + 165 + 129 = 454
s[1] (Lee)	{0xA0, 0xED, 0x81}	0xA0, 0xA5, 0x81	160 + 165 + 129 = 454
최종 결과 (result)			908

28. 다음 C 언어로 구현된 프로그램을 분석하여 그 실행 결과를 쓰시오. [24-1](#)

```
#include <stdio.h>
#include <stdlib.h>
main(int argc, char *argv[]) {
    int v1 = 0;
    int v2 = 35;
    int v3 = 29;
    if (v1 > v2 ? v2 : v1)
        v2 = v2 << 2;
    else
        v3 = v3 << 2;
    printf("%d", v2 + v3);
    return 0;
}
```

답: 151

[해설]

35+(29*4)=35+116=151

30. 다음 C 언어로 구현된 프로그램을 분석하여 그 실행 결과를 쓰시오. [24-1](#)

```
#include <stdio.h>
#include <ctype.h>
int main() {
    char *p = "It is 8";
    char result[100];
    int i;
    for(i = 0; p[i] != '\0'; i++) {
        if(isupper(p[i]))
            result[i] = (p[i] - 'A' + 5) % 25 + 'A';
        else if(islower(p[i]))
            result[i] = (p[i] - 'a' + 10) % 26 + 'a';
        else if(isdigit(p[i]))
            result[i] = (p[i] - '0' + 3) % 10 + '0';
        else if(!(isupper(p[i]) || islower(p[i]) || isdigit(p[i])))
            result[i] = p[i];
    }
    result[i] = '\0';
    printf("변환된 문자열 : %s\n", result);
    return 0;
}
```

답: 변환된 문자열 : Nd sc 1

35. 다음 C 언어로 구현된 프로그램을 실행시킨 결과가 "43215"일 때, <처리조건>을 참고하여 괄호에 들어갈 알맞은 식을 쓰시오. [23-2](#)

```
#include <stdio.h>
main() {
    int n[] = { 5, 4, 3, 2, 1 };
    for (int i = 0; i < 5; i++)
        printf("%d", (      ));
}
```

<처리조건>

괄호의 식에 사용할 문자는 다음으로 제한한다.

- n, i
- +, -, /, *, %
- 0~9, (,), [,]

답: `n[(i + 1) % 5]`

40. 다음 C 언어로 구현된 프로그램을 분석하여 그 실행 결과를 쓰시오. ★[23-2](#)

```
#include <stdio.h>
#define MAX_SIZE 10
int isWhat[MAX_SIZE];
int point = -1;

int isEmpty() {
    if (point == -1) return 1;
    return 0;
}

int isFull() {
    if (point == 10) return 1;
    return 0;
}

void into(int num) {
    if (isFull() == 1) printf("Full");
    else isWhat[++point] = num;
}

main() {
    into(5); into(2);
    while (!isEmpty()) {
        printf("%d", take());
        into(4); into(1); printf("%d", take());
        into(3); printf("%d", take()); printf("%d", take());
        into(6); printf("%d", take()); printf("%d", take());
    }
}
```

답: `213465`

2. 다음 Java로 구현된 프로그램을 분석하여 그 실행 결과를 쓰시오. [22-1](#)

```
class A {
```

```

    int a;
    int b;
}

public class Test {
    static void func1(A m) {
        m.a *= 10;
    }
    static void func2(A m) {
        m.a += m.b;
    }
    public static void main(String args[]) {
        A m = new A();
        m.a = 100;
        func1(m);
        m.b = m.a;
        func2(m);
        System.out.printf("%d", m.a);
    }
}

```

답: 2000

8. 다음 Java로 구현된 프로그램을 분석하여 그 실행 결과를 쓰시오. [20-4](#)

```

class Parent {
    int compute(int num) {
        if(num <= 1) return num;
        return compute(num - 1) + compute(num - 2);
    }
}
class Child extends Parent {
    int compute(int num) {
        if(num <= 1) return num;
        return compute(num - 1) + compute(num - 3);
    }
}
public class Test {
    public static void main(String[] args) {
        Parent obj = new Child();
        System.out.print(obj.compute(4));
    }
}

```

답: 1

[해설]

-1 -3 적용

10. 다음 JAVA로 구현된 프로그램을 분석하여 그 실행 결과를 쓰시오. [25-3](#)

```

enum Tri {
    A("A"), B("AB"), C("ABC");
    private String code;
    Tri(String code) {
        this.code = code;
    }
}

```

```

    }
    public String code() {
        return code;
    }
}

public class Main {
    public static void main(String[] args) {
        Tri t = Tri.values()[Tri.A.name().length()];
        System.out.print(t.code());
    }
}

```

답: **AB**

12. 다음 Java로 구현된 프로그램을 분석하여 괄호에 들어갈 알맞은 코드를 쓰시오. ★[25-3](#)

```

interface Cals {
    public void get(int v);
}

class Test (      ) Cals {
    public void get(int v) {
        System.out.print(v*v);
    }
}

public class Main {
    public static void main(String[ ] args) {
        Cals a = new Test();
        a.get(10);
    }
}

```

답: **implements**

13. 다음 Java로 구현된 프로그램을 분석하여 괄호에 들어갈 알맞은 코드를 쓰시오. [25-3](#)

```

class Rectangle {
    int x, y;
    Rectangle(int x, int y) {
        this.x = x;
        this.y = y;
    }
    int getArea( ) {
        return x*y;
    }
}

class Square extends Rectangle {
    Square(int a) {
        ( ㉠ )(a, a);
    }
    int getSquareArea( ) {
        return x*y;
    }
}

public class Main {
    public static void main(String[ ] args) {

```

```

        Square sq = new Square(10);
        System.out.println(sq.getSquareArea( ));
    }
}

```

답: **super**

14. 다음 Java로 구현된 프로그램을 분석하여 그 실행 결과를 쓰시오. [25-2](#)

```

public class Main {
    public static void change(String[ ] data, String s) {
        data[0] = s;
        s = "Z";
    }
    public static void main(String[ ] args) {
        String data[ ] = { "A" };
        String s = "B";
        change(data, s);
        System.out.print(data[0] + s);
    }
}

```

답: **BB**

15. 다음 Java로 구현된 프로그램을 분석하여 그 실행 결과를 쓰시오. [25-2](#)

```

public class Main {
    static interface F {
        int apply(int x) throws Exception;
    }
    public static int run(F f) {
        try {
            return f.apply(3);
        } catch (Exception e) {
            return 7;
        }
    }
    public static void main(String[ ] args) {
        F f = (x) -> {
            if (x > 2) {
                throw new Exception();
            }
            return x * 2;
        };
        System.out.print(run(f) + run((int n) -> n + 9));
    }
}

```

답: **19**

[해설]

Java의 람다식: 메소드 이름, 반환 타입, 인수 타입 없이 간단하게 **미리 선언된 함수형 인터페이스를 구현할 수 있는 것**으로, 코드를 더 짧고 읽기 쉽게 만들어주는 형식

<pre>int max(int a, int b) { return a > b ? a : b; }</pre>	->	<pre>(a, b) -> a > b ? a : b</pre>
---	----	--

```
}
```

Java 함수형 인터페이스와 랴다식(Lambda), 예외 처리 흐름 분석

- 함수형 인터페이스 F는 정수 인자 x를 받아 정수를 반환하는 추상 메서드 apply를 가집니다. 메서드 run(F f)은 전달받은 랴다식에 항상 값으로 3을 넣어 실행합니다.
- 첫 번째 호출인 run(f) 분석: 참조 변수 f에 대입된 랴다식은 입력값 x가 2보다 크면 예외(Exception)를 의도적으로 발생시킵니다. run 메서드 내부에서 f.apply(3)이 호출되면 조건문 $3 > 2$ 가 참이 되므로 강제로 예외가 던져집니다.
- 이로 인해 예외가 발생하여 run 메서드의 catch (Exception e) 블록으로 제어가 이동하고, 상숫값 7이 반환됩니다.
- 두 번째 호출인 run((int n) -> n + 9) 분석: 예외 발생 조건이 없는 새로운 익명 함수가 전달됩니다. run 메서드 내부에서 값 3을 대입해 apply(3)을 실행하므로 연산식은 $3 + 9$ 가 되어 정상적으로 12를 반환합니다.
- 최종적으로 System.out.print()에 의해 두 반환값의 덧셈 연산인 $7 + 12 = 19$ 가 출력됩니다.

호출 구문	내부 실행 및 예외 여부	반환값
run(f)	x=3 대입 → 조건 참, 예외 발생 → catch 진입	7
run((int n) -> n + 9)	n=3 대입 → 3 + 9 정상 연산 실행	12

17. 다음 Java로 구현된 프로그램을 분석하여 그 실행 결과를 쓰시오. 25-2

```
public class Main {
    public static class BO {
        public int v;
        public BO(int v) {
            this.v = v;
        }
    }
    public static void main(String[] args) {
        BO a = new BO(1);
        BO b = new BO(2);
        BO c = new BO(3);
        BO[] arr = {a, b, c};
        BO t = arr[0];
        arr[0] = arr[2];
        arr[2] = t;
        arr[1].v = arr[0].v;
        System.out.println(a.v + "a" + b.v + "b" + c.v);
    }
}
```

답: 1a3b3

19. 다음 Java로 구현된 프로그램을 분석하여 그 실행 결과를 쓰시오. 25-1

```
public class Main {
    public static void main(String[] args) {
        new Child();
        System.out.println(Parent.total);
    }
}

class Parent {
    static int total = 0;
    int v = 1;
    public Parent() {
        total += ++v;
    }
}
```



```

        show();
    }
    public void show() {
        total += total;
    }
}
class Child extends Parent {
    int v = 10;
    public Child() {
        v += 2;
        total += v++;
        show();
    }
    @Override
    public void show() {
        total += total * 2;
    }
}

```

답: 54

[해설]

상속 구조에서의 다형성(오버라이딩) 및 static 변수 값의 시계열 변화 연산 과정

• 인스턴스 생성 시작 (new Child());:

자식 클래스의 객체가 생성될 때, 자바 규칙에 따라 부모 클래스인 Parent의 생성자가 기본적으로 먼저 내부 호출됩니다.

• 부모 클래스 생성자 연산 블록 실행:

- total += ++v; 연산 수행: Parent 내 인스턴스 변수 v가 1에서 2로 먼저 전위 증가합니다. 결과적으로 공유 정적 변수인 total = 0 + 2 = 2 상태가 됩니다.
- show(); 메서드 호출 (★핵심): 현재 생성 중인 실제 인스턴스는 자식 객체인 Child이므로, 자바의 동적 바인딩 규칙에 의해 부모의 메서드가 아닌 자식 클래스에서 재정의(Override)한 Child.show()가 대신 실행됩니다.
- Child.show() 내용 연산: total += total * 2; 이므로 total = 2 + (2 * 2) = 6입니다.

• 자식 클래스(Child) 생성자 연산 블록 실행:

- v += 2; 연산 수행: Child의 고유 변수 v는 10이었으므로 10 + 2 = 12가 됩니다.
- total += v++; 연산 수행: 후위 증가 연산이므로 현재 v의 값인 12가 먼저 반영됩니다. 즉, total = 6 + 12 = 18이 되고, 연산 완료 직후 v는 13이 됩니다.
- show(); 메서드 호출: 오버라이딩된 Child.show()가 명확하게 실행됩니다. total += total * 2; 규칙에 맞춰 연산하면 total = 18 + (18 * 2) = 18 + 36 = 54가 최종 출력됩니다.

Parent 클래스		Child 클래스
total	v	v
0	1	10
2	2	12
6		13
18		
54		

20. 다음 Java로 구현된 프로그램을 분석하여 그 실행 결과를 쓰시오. 25-1

```

public class Main {
    public static void main(String[] args) {
        System.out.println(calc("5"));
    }
    static int calc(int value) {
        if (value <= 1) return value;
        return calc(value - 1) + calc(value - 2);
    }
}

```

```
static int calc(String str) {
    int value = Integer.valueOf(str);
    if (value <= 1) return value;
    return calc(value - 1) + calc(value - 3);
}
```

답: 4

[해설]

첨에는 -1 -3, 나머지는 -1, -2

21. 다음 Java로 구현된 프로그램을 분석하여 그 실행 결과를 쓰시오. [25-1](#)

```
public class Main {
    public static void main(String[] args) {
        int[] data = {3, 5, 8, 12, 17};
        System.out.println(func(data, 0, data.length - 1));
    }
    static int func(int[] a, int st, int end) {
        if (st >= end) return 0;
        int mid = (st + end) / 2;
        return a[mid] + Math.max(func(a, st, mid), func(a, mid + 1, end));
    }
}
```

답: 20

[해설]

$8 + \text{Math.max}(\text{func}(0,2), \text{func}(3,4)) \rightarrow 8 + \text{Math.max}(8, 12) \rightarrow 8 + 12 = 20$ 이 됩니다.

함수 호출 구간	mid 인덱스	a[mid] 값	최종 반환(Return)값
func(0, 1)	0	3	$3 + \text{Math.max}(0, 0) = 3$
func(0, 2)	1	5	$5 + \text{Math.max}(3, 0) = 8$
func(3, 4)	3	12	$12 + \text{Math.max}(0, 0) = 12$
func(0, 4) [최상위]	2	8	$8 + \text{Math.max}(8, 12) = 20$

22. 다음 Java로 구현된 프로그램을 분석하여 그 실행 결과를 쓰시오. [24-3](#)

```
public class Main {
    static String[] x = new String[3];
    static void func(String[] x, int y) {
        for(int i = 1; i < y; i++) {
            if(x[i-1].equals(x[i])) {
                System.out.print("O");
            }
            else {
                System.out.print("N");
            }
        }
        for(String z : x) {
            System.out.print(z);
        }
    }
}
```

```

public static void main(String[] args) {
    x[0] = "A";
    x[1] = "A";
    x[2] = new String("A");
    func(x, 3);
}
}

```

답: **OOAAA**

[해설]

Java의 문자열 비교 및 배열 순회 분석

- main 메서드에서 전역 배열 x의 각 요소에 문자열 "A"를 저장합니다. 리터럴 방식 생성과 new String() 방식 생성이 섞여 있지만, 모두 내부에 저장된 문자열 자체는 "A"로 동일합니다.
- func(x, 3)이 호출되면서 반복문이 i = 1부터 i < 3까지 수행됩니다.
- Java에서 문자열의 **값 자체를 비교할 때는** 주소값 비교(==)가 아닌 equals() 메서드를 사용합니다. 따라서 두 번의 조건문 비교 모두 참이 됩니다.
 - i = 1 일 때: x[0].equals(x[1]) → "A".equals("A")이므로 참 → "O" 출력
 - i = 2 일 때: x[1].equals(x[2]) → "A".equals("A")이므로 참 → "O" 출력
- 첫 번째 루프가 종료되면 출력 결과는 **OO**가 됩니다.
- 그 후 향상된 for문을 통해 배열 x의 모든 요소(x[0], x[1], x[2])인 **"AAA"**를 순서대로 이어서 **OOAAA**가 출력됩니다.

x[0]	x[1]	x[2]	y	i	z
"A" (Literal)	"A" (Literal)	"A" (new String)	3	1	A
x[0].equals(x[1]) → True ('O')		-		2	A
-	x[1].equals(x[2]) → True ('O')			3	A

23. 다음 Java로 구현된 프로그램을 분석하여 그 실행 결과를 쓰시오. [24-3](#)

```

public class Main {
    public static void main(String[] args) {
        B a = new D();
        D b = new D();
        System.out.print(a.getX() + a.x + b.getX() + b.x);
    }
}
class B {
    int x = 3;
    int getX() {
        return x * 2;
    }
}
class D extends B {
    int x = 7;
    int getX() {
        return x * 3;
    }
}

```

답: **52**

[해설]

21 + 3 + 21 + 7 = 52

25. 다음 Java로 구현된 프로그램을 분석하여 그 실행 결과를 쓰시오. ★[24-3](#)

```

class Printer {
    void print(Integer a) { System.out.print("A" + a); }
    void print(Object a) { System.out.print("B" + a); }
    void print(Number a) { System.out.print("C" + a); }
}
public class Main {
    public static void main(String[] args) {
        new Collection<>().print();
    }
    public static class Collection<T> {
        T value;
        public Collection(T t) { value = t; }
        public void print() {
            new Printer().print(value);
        }
    }
}

```

답: B0

28. 다음 Java로 구현된 프로그램을 분석하여 그 실행 결과를 쓰시오. [24-2](#)

```

interface Number {
    int sum(int[] a, boolean odd);
}
public class Test {
    public static void main(String[] args) {
        int a[] = {1, 2, 3, 4, 5, 6, 7, 8, 9};
        OENumber OE = new OENumber();
        System.out.print(OE.sum(a, true) + ", " + OE.sum(a, false));
    }
}
class OENumber implements Number {
    public int sum(int[] a, boolean odd) {
        int result = 0;
        for (int i = 0; i < a.length; i++) {
            if ((odd && a[i] % 2 != 0) || (!odd && a[i] % 2 == 0)) {
                result += a[i];
            }
        }
        return result;
    }
}

```

답: 25, 20

32. 다음 JAVA로 구현된 프로그램을 분석하여 그 실행 결과를 쓰시오. ★[23-3](#)

```

class SuperObject {
    public void draw() {
        System.out.println("A");
        draw();
    }
    public void paint() {

```

```

        System.out.print('B');
        draw();
    }
}
class SubObject extends SuperObject {
    public void paint() {
        super.paint();
        System.out.print('C');
        draw();
    }
    public void draw() {
        System.out.print('D');
    }
}
public class Test {
    public static void main(String[ ] args) {
        SuperObject a = new SubObject();
        a.paint();
        a.draw();
    }
}

```

답: **BDCDD**

33. 다음 JAVA로 구현된 프로그램을 분석하여 그 실행 결과를 쓰시오. [23-3](#)

```

class P {
    public int calc(int n) {
        if (n <= 1) return n;
        return calc(n - 1) + calc(n - 2);
    }
}
class C extends P {
    public int calc(int n) {
        if (n <= 1) return n;
        return calc(n - 1) + calc(n - 3);
    }
}
public class Test {
    public static void main(String[ ] args) {
        P obj = new C();
        System.out.print(obj.calc(7));
    }
}

```

답: **2**

[해설]

-1, -3

34. 다음 Java로 구현된 프로그램의 실행 순서를 나열하시오. ★(단, 같은 번호는 중복해서 작성하지 마시오.) [24-1](#)

```

class Parent {
    int x, y;
}

```

```

1 Parent(int x, int y) {
    this.x = x;
    this.y = y;
}
2 int getX() {
    return x*y;
}
}

class Child extends Parent {
    int x;
3 Child(int x) {
    super(x+1, x);
    this.x = x;
}
4 int getX(int n) {
    return super.getX() + n;
}
}

public class Main {
5 public static void main(String[] args) {
6     Parent parent = new Child(10);
7     System.out.println(parent.getX());
}
}

```

답: 5 → 6 → 3 → 1 → 7 → 2

36. 다음 Java로 구현된 프로그램을 분석하여 그 실행 결과를 쓰시오. [23-1](#)

```

class Static {
    public int a = 20;
    static int b = 0;
}

public class Test {
    public static void main(String[] args) {
        int a = 10;
        Static.b = a;
        Static st = new Static();
        System.out.println(Static.b++);
        System.out.println(st.b);
        System.out.println(a);
        System.out.print(st.a);
    }
}

```

답: 10
11
10
20

5. 다음 Python 프로그램과 그 <실행결과>를 분석하여 괄호에 들어갈 알맞은 예약어를 쓰시오.
(<실행결과> 첫 번째 라인의 '5 10'은 입력받은 값에 해당한다.) [23-3](#)

```

x, y = input("x, y의 값을 공백으로 구분하여 입력 : ").(      )(' ')

print("x의 값 : ", x)
print("y의 값 : ", y)

```

<실행결과>

```
x, y의 값을 공백으로 구분하여 입력 : 5 10
x의 값 : 5
y의 값 : 10
```

답: **split**

11. 다음 Python으로 구현된 프로그램을 분석하여 그 실행 결과를 쓰시오. [25-2](#)

```
lst = [1,2,3]
dst = {i : i * 2 for i in lst}
s = set(dst.values( ))
lst[0] = 99
dst[2] = 7
s.add(99)
print(len(s & set(dst.values( ))))
```

답: **2**

[해설]

{2, 4, 6, 99} & {2, 6, 7} = {2, 6} 길이(len)는 교집합의 원소 개수이므로 2가 됩니다.

12. 다음 Python으로 구현된 프로그램을 분석하여 그 실행 결과를 쓰시오. [25-1](#)

```
class Node:
    def __init__(self, value):
        self.value = value
        self.children = []

def tree(li):
    nodes = [Node(i) for i in li]
    for i in range(1, len(li)):
        nodes[(i - 1) // 2].children.append(nodes[i])
    return nodes[0]

def calc(node, level=0):
    if node is None:
        return 0
    return (node.value if level % 2 == 1 else 0) + sum(calc(n, level + 1) for n in node.children)

li = [3, 5, 8, 12, 15, 18, 21]
root = tree(li)
print(calc(root))
```

답: **13**

[해설]

이 문제는 주어진 리스트를 완전 이진 트리(Complete Binary Tree) 구조로 구성한 후, 탐색 깊이(level)가 홀수(Level 1)일 때만 노드의 값을 더하고, 짝수(Level 0, 2)일 때는 0을 반환하는 재귀 함수 알고리즘입니다.



Level 0 루트 노드 연산 (3)

- level = 0 (짝수)이므로 루트 노드의 값 3 대신 0을 취합니다.

- 최종 연산 식: 0 + 5 + 8 = 13

13. 다음 Python으로 구현된 프로그램을 분석하여 그 실행 결과를 쓰시오. [24-3](#)

```
def func(lst):
    for i in range(len(lst) // 2):
        lst[i], lst[-i-1] = lst[-i-1], lst[i]

lst = [1,2,3,4,5,6]
func(lst)
print(sum(lst[::2]) - sum(lst[1::2]))
```

답: 3

[해설]

함수 func(lst)를 통해 리스트의 요소를 **역순(Reverse)**으로 뒤집은 후, 짝수 인덱스 요소들의 합 - 홀수 인덱스 요소들의 합 =

1. func(lst) 실행 후 리스트 역순 변경 :



12-9 = 3